

Donghyung Lee

DevOps Engineer | Resume & Career Description · 4Y 3M Experience

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94

Introduction

Hello, I am Donghyung Lee, a DevOps Engineer.

Aiming for stable infrastructure operation, I codify every change as IaC and turn incident response into postmortem-driven assets to cut cloud costs 20-50% while raising operational efficiency. My core strength is interrogating the purpose of each task and pursuing a better direction.

I design and build infrastructure across AWS and GCP cloud and an OpenStack-based on-premises PaaS. I standardize container operations with Kubernetes, codify infrastructure with Terraform and Ansible, and run GitLab CI / ArgoCD-centered deploy pipelines alongside monitoring and logging stacks. Repetitive work is automated in Python and Bash.

I also operate adjacent data-engineering and AI/ML ops through data-pipeline automation and an in-house GPU LLM service, and build in-house DevOps tools myself — git-bridge, a Slack APK bot, and more.

Across cloud and on-premises, I solely led AWS → GCP migration, AWS + on-premises hybrid, and on-premises PaaS builds — achieving 20% AWS and 50% GCP cost cuts, 50-67% faster deploys, and 90%+ manual-work automation. Most recently I built an AWS EKS multi-cluster for a new game's cloud production launch, migrated helmfile-push deploys to a more stable auto-syncing model (ArgoCD pull-GitOps), and — alongside resolving 14-second game-DB stalls and unifying login via Keycloak central SSO — raised operational efficiency another notch.

I aim to keep contributing to infrastructure architecture advancement and organizational productivity.

Strengths

Asking 'Why' First

- Habit of asking 'why' first — define the business problem before picking the tool

Systematic Documentation & Knowledge Sharing

- Document and share every task, building organizational knowledge assets

Record-Based Continuous Improvement

- Postmortem-record every incident, infra change, and troubleshooting — preventing recurrence, driving improvement

Full-Stack Infrastructure Experience

- End-to-end infra across cloud (AWS, GCP), on-premises, networking, CI/CD, and monitoring

Skills

Cloud	AWS, GCP, OpenStack
Container Orchestration	Kubernetes, Docker, Cilium CNI, NGINX Gateway Fabric / Gateway API, Karpenter
Infrastructure as Code	Terraform, Ansible, Helm, Helmfile
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, ArgoCD ApplicationSet, Jenkins, GitOps
Monitoring & Observability	PLG Stack (Prometheus/Loki/Grafana), ELK Stack, EFK Stack, ECK Operator, Fluent Bit, Thanos, OpenTelemetry, Grafana Tempo
Security & IAM	Keycloak (Operator, OIDC IdP), Vaultwarden, GitLab SSO (OIDC), External Secrets Operator, Sealed Secrets
Automation	Go, Python, Shell Scripting
Database	MySQL, PostgreSQL, Redis, MongoDB
OS & Servers	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration, savings reinvested in new projects)
- Kubernetes storage performance optimization (control-plane VM SSD migration brought etcd fsync and apiserver latencies back to normal; GitLab Runner build I/O isolation recovered game DB COMMIT latency from 14s back to ms)

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- 50-67% deployment time reduction with continuous GitOps deployment shortening Time to Market, 95% rollback time decrease (30min → 1-2min)
- Built a layered shared toolchain image architecture + auto version cycle standardizing every consumer repo — first-job install time 5min → ~10s (~95% reduction), with 4-stage cascade automated by just 2 manual clicks

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Evolved from ELK → EFK Stack, then redesigned into ECK Operator-based Elastic Stack 9.0 CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- Fluent Bit data integrity & HA hardening (SQLite offset DB + PVC, PodDisruptionBudget + preStop hook, 7 Prometheus alerts — zero log gap / duplication on Pod restart)

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- Migrated shell → Python (stdlib only, 758 unittests) + wave-based auto-deploy across 25 components + auto-upgrade MR generation
- Established self-sufficient Kubernetes ops at DP (Delivery Partner) sites — designed and ran a 6-month training program (avg 10 attendees, 4h/session), standardized materials reused across other clients

Kubernetes Platform Modernization & Open-Source Contribution

Zero-downtime migration of network/logging stacks and releasing reusable public charts as open source

- Migrated multiple ingress-nginx instances to a single NGINX Gateway Fabric control-plane + per-class Gateway CRs with zero downtime (full cutover, wildcard TLS consolidated)
- Released 3 open-source Helm charts (nginx-gateway-cr, elasticsearch-eck, kibana-eck) and multiple composite GitHub Actions on ArtifactHub / GitHub Marketplace

Infrastructure Security & IAM

Centralized in-house authentication and secret governance via Keycloak Operator-based SSO and Vaultwarden password management

- Introduced central SSO with the Keycloak Operator + KeycloakRealmImport — let existing GitLab accounts keep logging in, integrated ArgoCD / Harbor / Vaultwarden OIDC (zero downtime, verification checks PASS), and established a single authorization flow from GitLab groups → Keycloak mappers → ArgoCD roles
- Deployed Vaultwarden on Kubernetes via Helm with GitLab SSO + automated backups (scheduled SQLite dumps) + interactive restore script — centralized management for 70 users and 50 secrets

Data Engineering & Internal LLM Services

End-to-end ownership from GCP data pipelines to in-house LLM services

- Built a multi-source (MongoDB · CloudSQL · GA · Dune) → BigQuery → Google Sheets pipeline by combining BigQuery, Dataflow, Cloud Functions, and Cloud Scheduler — entire infrastructure 100% Terraform IaC
- Deployed an internal LLM service (Ollama + Open WebUI) on a Kubernetes-hosted on-premises GPU server, saving ~\$100/month by replacing external AI SaaS subscriptions and ensuring 100% internal security compliance

Professional Experience (Detailed)

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture that reduced monthly infrastructure costs by 20% (\$60K → \$48K) and channeled the savings into new project investments.

Currently providing secondary technical support and operations for a live-service game in collaboration with the external publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation, in collaboration with the external publisher's ops team) 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of \$60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud auto-scaling unnecessary, while only QA/Production required scalability and stability. Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS.

The two environments are operationally independent without VPN (application traffic isolated); the image registry is also split by environment (dev = Harbor, QA/Production = ECR), with each environment building independently to match its architecture. Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage for new components, IAM excluded) across both environments, achieving consistent infrastructure management without increasing operational complexity.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version.
Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- During rolling deploys of the ArgoCD-managed production services, the ALB target deregistration delay didn't line up with Pod SIGTERM handling, dropping some session connections.
Achieved zero-downtime deploys with 0 connection drops via 3 changes — (1) aligned ALB drain timing: a preStop hook + readiness-first fail block new requests during termination while in-flight requests are served to completion, (2) tuned the RollingUpdate strategy surge-first to fit replica scale, holding capacity through rollout, (3) applied a PDB to multi-replica services so Karpenter consolidation never evicts Pods simultaneously

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% (\$60,000 → \$48,000), reinvesting ~\$144,000 annual savings into new project infrastructure for resource reinvestment cycle
 - Offloaded dev servers to on-premises, cutting ~\$1,000/month in AWS resource costs and optimizing resource usage
 - Ensured operational stability and fault isolation through environment role separation
-

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD.

Built Docker images with Kaniko without Docker-in-Docker (no privileged container), and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- Applying `selfHeal=true` uniformly risked prod changes auto-applying without human approval → split by environment: prod uses `selfHeal=false` + manual sync windows to gate deploys, dev/staging keep `selfHeal=true` for automatic drift correction, with every sync event wired to Slack for change traceability, stability, and emergency-response flexibility
- Moved data upload from direct SSH scp/rsync to S3 transit-bucket staging → SSM SendCommand-based `aws s3 sync --exact-timestamps`, removing SSH keys and deploy-account key-leak risk

Results

- 67% deployment time reduction (30min → 10min), and cut the data-deployment source fetch from ~139s to ~30s by replacing it with `git fetch --depth=1 --filter=blob:none`
- Split deployment via GitOps automation — dev/qa/review run continuous merge-driven deployment (~100/week per core service repo) to cut new-feature Time to Market, while prod ships controlled ~5-7 releases/week gated behind manual sync windows for stability
- Replaced the manual build/deploy/config-apply steps with the pipeline to eliminate human-error deployment failures, and the cross-repo MasterData pipeline auto-fans-out to downstream services on data changes while preventing circular (loop) triggering

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present

Starting from a log-scattered on-premises environment, advanced centralized logging through three stages (ELK → EFK → ECK Operator), hardened collector data integrity / HA, then repurposed the accumulated EFK log pipeline into a game user-retention analytics platform. Solely designed and operated the full observability lifecycle — build → operate → improve → analytics.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

With server/container logs scattered on-premises — slow root-cause and no unified monitoring — enhanced the logging stack in **three stages: ELK → EFK → ECK Operator**. Built centralized logging on ELK first, then migrated to an EFK stack (lightweight Fluent Bit DaemonSet collection + Fluentd processing) to resolve Logstash's JVM memory burden and scaling limits. As Elastic's official Helm chart stalled for over two years, moved to CRD-based declarative management (ECK Operator), performing a zero-downtime major upgrade of Elastic Stack from 8.5.1 to 9.0.0, and templated the on-premises EFK stack into AWS(EKS) `-aws` stacks to port the same model with keyless IRSA (ILM, S3 snapshot retention).

Results: 90% log-search time reduction & improved MTTR, ECK migration removed manual TLS/account/upgrade burden (single-line CR spec.version rolling upgrades), and completed a stalled 2-year Elastic Stack major upgrade with zero downtime (8.5.1 → 9.0.0).

3-2. Fluent Bit Log Collector Data Integrity & HA Hardening (2026.03 ~ 2026.05)

Solved log re-index/loss risk from Fluent Bit tail-offset loss on Pod restart with a SQLite offset DB and dedicated PVC — zero re-index / zero duplicates, 7 alerts pre-empting silent collector failures

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

Extended a search-only EFK stack into a product-analytics platform via an Elasticsearch Transform (5-min) pivoting events to one row per user — a 12-panel Kibana dashboard delivering D+1~D+30 retention and cohorts, no separate SaaS

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% 2025.02 ~ Present

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Deployed a Python bot on Kubernetes auto-sending completion messages and download QR codes to Slack on Jenkins APK build finish — 90% QA delivery-time reduction, version confusion eliminated

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

Integrated GitLab Webhook with Google Drive API to auto-convert markdown into Google Docs on push, optimized with rclone `--checksum` — 80% doc-management reduction (10min→2min), GitLab unified as SSOT

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Deployed Ollama and Open WebUI on Kubernetes on an on-prem GPU server with NFS model storage — 15 developers running ~100 queries/day, saving ~\$100/month over external SaaS and removing data-leak risk

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ 2026.06)

Developed and open-sourced a Go-based bidirectional Git mirroring tool (git-bridge), fully automating sync across CodeCommit/GitLab/GitHub for 10 repos with ~30 sync events/day.

Authenticated a `POST /retry/mirror` endpoint with a dedicated token `RETRY_API_TOKEN` (constant-time compared), separated from the webhook secret, for a recovery path that never exposes the secret, and blocked non-fast-forward regressions with `ref_overrides` pinning sync direction per ref pattern plus a **fail-open** fallback (6/6 PASS on a test-only repo, production isolated).

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

Developed a lightweight Go-based static file server (static-file-server) in-house, implementing the operational features it needed, and operated a Helm repository on GitHub Pages to establish a deployment standard.

Deployed to Kubernetes via an in-house Helm chart (NFS storage), completing a full replacement of halverneus/static-file-server, handling ~30

downloads/day and 5 APK builds/week, and released as open source under Apache-2.0 so security patches and new features run on an in-house cycle.

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% (Solo design — monitoring · upgrade automation · Helm management framework · NGF migration · 3 OSS Helm charts · storage performance optimization · Shell→Python migration; captured as 4 automation scripts establishing a standard procedure → so the team can reuse) 2025.12 ~ Present

To advance cluster visibility, operational stability, network modernization, storage performance, and automation-code quality, overhauled monitoring, automated K8s upgrades and Helm management, and migrated the network controller onto the Gateway API. Released the artifacts from the NGF migration and ECK Operator adoption as OSS Helm charts, resolved storage bottlenecks via control-plane / DB-node SSD migration and build I/O isolation, and modernized the infra deploy/upgrade pipeline with wave-based automation + a Shell → Python migration.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Designed custom alerts and Grafana dashboards on kube-prometheus-stack, integrating DB, node-exporter, and Cilium metrics — 27 custom alert rules across 9 component groups, severity routing and inhibit rules removing duplicates

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Built a Kubespray upgrade automation script and a multi-step health check (node/etcd/API Server), standardizing version management with a Helm framework and 4-canonical-template synchronizer — 90% verification-time reduction, automated cert renewal removing expiration outages

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Migrated the on-prem ingress-nginx fleet to a single NGF 2.x control-plane + per-class Gateway CRs — MetalLB IPs preserved so DNS/firewall unchanged, incident-free cutover (30-60s per class), one wildcard cert cutting manual renewal 50%

5-4. Three Open-Source Helm Charts — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

Refined NGF and ECK outputs into three reusable charts (nginx-gateway-cr·elasticsearch-eck·kibana-eck), published on ArtifactHub under Apache-2.0 — cut the mgmt cluster over to public OCI charts with cluster_uuid unchanged and zero downtime

5-5. Storage Performance Optimization — etcd / DB SSD Migration + Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

The control-plane VM and the game-DB worker VM both ran on HDDs, contending with GitLab Runner buildx for I/O, so etcd WAL fsync p99 ballooned to ~2s, apiserver GET p99 to ~7s, and game DB COMMIT latency to 9~14s — causing recurring play-blocking incidents. Tuned MySQL fsync parameters to immediately ease the disk fsync load, then migrated the control-plane VM to SSD with 8.5 minutes of downtime (vs. a 15-minute estimate), and losslessly migrated the game-DB worker VM's MySQL and Redis via backup → SSD migration → restore. Finally, stood up a dedicated build-only VM on a separate physical server and isolated buildx via taint/toleration to structurally prevent I/O contention.

Results: diagnosed GitLab Runner buildx disk-I/O contention as the root cause and migrated the control-plane VM to SSD, bringing etcd fsync and apiserver latencies — which had spiked into the seconds range — back to normal; **game DB COMMIT recovered from 9~14s to the ms range**, taking play-blocking incidents during builds to zero, and a dedicated build VM physically isolated buildx from DB workloads.

5-6. Infra Deploy/Upgrade Automation + Shell → Python Migration (2026.03 ~ Present)

The internal Kubernetes infra monorepo's 25 components were manually managed by helmfile + shell, hitting limits on multi-env branching and complex YAML patching. Structured the CI pipeline into 6 waves (bootstrap → core → security → db-redis → observability → apps), detecting MR diffs to helmfile diff/apply only changed components with a manual gate standardizing sequential deploys, and automated upgrades so CI detects new versions across the 25 components and opens an auto-MR rendering changed chart, values diff, and breaking-change notes. Migrated the existing shell + awk automation to Python (stdlib only), modularizing canonical templates, sync/backup, and CI scripts with 758 unittests, and adopting Project 7's helmfile-tools image achieved **CI bootstrap time from 5min to ~10s (~95% reduction)**. This helmfile-push deploy model was subsequently migrated to the ArgoCD pull-GitOps control plane in 5-7 (deploy-model evolution).

5-7. helmfile Push → ArgoCD Pull-GitOps Migration (App-of-apps Control Plane, 2026.06 ~ Present)

Platform releases deployed via helmfile push tied the deploy actor to the CI runner, and scaling to multiple clusters called for a pull-based GitOps control plane to declaratively converge per-cluster drift. A Matrix + Git-files generator ApplicationSet (auto-emitting controller) reads each component's `argocd/*.yaml` markers to create Applications, registering 16 on-premises + 8 AWS = 24 platform releases across 2

clusters.

Zero-downtime adoption: existing helm releases were handed over to ArgoCD without re-creation (adopt), preserving deploy order.

Stabilization / least privilege: App-of-apps prune-cascade guardrails block mass deletion, and least-privilege AppProjects shrink blast radius.

Results: 24 releases across 2 clusters (16 on-premises + 8 AWS) declaratively managed from a single control plane, re-creation-free (zero-downtime) adoption for a zero-downtime cutover, and prune-cascade + least-privilege AppProjects shrinking blast radius.

Project 6: Infrastructure Security System

Contribution 100% 2026.04 ~ Present

6-1. Vaultwarden Password Management System (1 week, 2026.04 ~ Present)

Centralized internal secrets scattered across Slack/email/personal vaults with Vaultwarden — deployed on Kubernetes via Helm with GitLab SSO (OIDC) and RBAC, migrating 70 users and 50 secrets at zero SaaS cost

6-2. Keycloak Operator-based Central SSO Introduction (2026.04 ~ Present)

ArgoCD / Harbor / Vaultwarden each used GitLab directly as their OIDC IdP, layering dependence on GitLab's user / session / MFA policy — consolidated onto a central login broker (Keycloak). Introduced the Keycloak Operator + KeycloakCR + KeycloakRealmImport with a PostgreSQL backend, brokering GitLab as Identity Provider so existing accounts keep logging in, codified Realm / Client / IdP / Group / Mapper via a bootstrap script, and brokered GitLab groups through Keycloak group mappers wired into ArgoCD roles to consolidate the authorization SSOT onto Keycloak.

Results: ArgoCD / Harbor / Vaultwarden OIDC integrations completed with zero downtime and verification checks PASS; captured five pitfalls hit during adoption (IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping) into the plan doc to prevent recurrence; laid the groundwork for future LDAP federation, MFA, and central session policy.

6-3. GitOps Secret Management — External Secrets Operator · Sealed Secrets (2026.05 ~ Present)

Replaced plaintext DB passwords in Git by wiring External Secrets Operator to AWS Secrets Manager for ExternalSecret references and standardizing the Harbor pull secret as a cluster-wide SealedSecret — making AWS SM the single source of truth

6-4. OpenVPN Remote-Access Server (2026.07)

Built an OpenVPN remote-access server in-house on a physical server to lift the iptime router's built-in VPN 5-concurrent-connection limit — easy-rsa PKI with per-client EC certificate (prime256v1) auth + tls-crypt encrypting and authenticating the control channel, AES-256-GCM / TLS 1.2+, full-tunnel by default with split-tunnel switchable, and the 10.8.0.0/24 tunnel range routed to the internal physical-server network via SNAT (MASQUERADE) (configured for up to 50 concurrent connections).

Automated operations with an idempotent bash install script (PKI build → server.conf render → NAT systemd unit → service start) and a client-certificate issue/revoke/CRL-refresh lifecycle script, controlled remotely from the admin Mac over SSH

Project 7: Shared Toolchain Image Layered Architecture + Auto Version Cycle

Contribution 100% (Architecture design · CI standardization · automation pipeline) 2025.09 ~ 2026.05

Problem

Across the 15 repos in the GitLab CI standardization scope (1 template provider + 14 consumers), each job re-installed helm / helmfile / kubectl / crane / aws-cli on top of an Alpine base, making first-job bootstrap take an average of ~5 minutes. Tool versions were scattered across the 14 consumer repos, making coordinated bumps effectively impossible.

Reliance on `:latest` tags also undermined reproducibility.

Approach

Layer 1 (Mirror): copied external registry images into the internal Harbor via `crane copy`, preserving multi-arch manifests.

Layer 2 (Custom): built 7 fat images on top of Layer 1 (helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools).

Tier 1~2 SSOT: introduced a shared `.toolchain_build_custom` template + central `TOOLCHAIN_*_TAG` variables, indirecting consumer repos away from tracking image tags themselves.

Shared CI template SSOT: extracted the CI jobs copy-pasted per repo into a central shared CI template repo, each repo referencing them via `include`. Manages the auth (keyless AWS auth) / build (kaniko) / deploy (ArgoCD) / backup (Google Drive) job templates plus shared anchors (git-ssh-rules:job-config-packages) from a single source, removing copy-paste drift.

Phase 4 auto cycle: a cron job drives the following 4 steps so the full cascade completes with just two manual clicks:

- (1) Upstream version detection
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 sync auto-MR
- (4) Auto-propagation to 14 consumer repos

Results

- **First-job bootstrap time cut from 5min to ~10s (~95% reduction)**, with 100% tool-version consistency across 14 consumer repos (15 repos total), compressing 12+ scattered version points into a 4-tier governance flow (ARG → central var → consumer indirection → lint drift check)
- The Phase 4 cron auto cycle lands new versions of helm / helmfile / kubectl / crane / rclone (5+ tools) with just 2 manual clicks, and a minor-guard policy (helm/helmfile/kubectl) prevents compatibility breakage in advance
- Built a risk-tiered auto-upgrade pipeline across 25 components (T1 weekly / T2 biweekly / T3 monthly + pre-flight) — version detection → auto-MR → Slack alert → wave-sequenced apply, with a MAJOR_PIN guard and T3 atomic rollback — and built amd64 + arm64 multi-arch images via docker buildx while mirroring external images into Harbor with crane, removing external-registry dependency

Project 8: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (Solo greenfield EKS design & build) 2026.06 ~ Present (Building & Operating)

Problem

Separately from the live game's estate (AWS QA-Prod), the new externally-published game ran only as a single on-premises development cluster, and launching it to production required a separate, scalable, highly-available AWS EKS production environment — on-premises alone could not handle global traffic and autoscaling needs.

Approach

Built a greenfield EKS cluster in eu-central-1, expanding the estate into an on-premises dev + AWS production multi-cluster hybrid, and managed ~11 platform components entirely via GitOps (ArgoCD).

Autoscaling / networking: Chose Karpenter over Cluster Autoscaler (unified spot across families, faster node provisioning) to run interruption-resilient multi-family spot autoscaling plus a dedicated on-demand NodePool for game servers, while AWS Load Balancer Controller (ALB/NLB) + ExternalDNS (Route53) + a wildcard ACM cert automate ingress, DNS, and TLS.

Logging / secrets: an eck-operator-based EFK stack (Elasticsearch 3-node 3-AZ, EBS gp3, S3 snapshot SLM) runs HA, with External Secrets Operator externalizing secrets; every ServiceAccount authenticates via IRSA / Pod Identity, removing static keys.

Auth / deploy: ArgoCD integrates GitLab OAuth SSO + game-scoped RBAC; client artifacts ship via Jenkins → S3 + CloudFront, and server manifests reach the bastion's EFS mount via an S3 transit bucket + SSM SendCommand — the bastion is discovered dynamically by Name tag, so deploys use no SSH keys.

Troubleshooting

- Game-production Pods reserved ~4x the memory they actually used, causing node sprawl → right-sized memory requests from a measured profile, normalizing scale-out and improving spot cost efficiency
- During rolling deploys, ALB routed traffic to not-yet-ready Pods → combined ALB Pod readiness gate + PodDisruptionBudget + graceful drain to secure zero-downtime deploys

Results

- Expanded from a single on-premises dev cluster to an on-premises + AWS production multi-cluster hybrid, establishing the cloud production launch foundation for the new externally-published game
- Managed ~11 platform components 100% declaratively via GitOps (ArgoCD), and established static-key-free authentication with IRSA / Pod Identity + SSM SendCommand keyless deployment, removing bastion SSH-key leak risk
- ArgoCD app markers plus a production CI/CD transition doc make this a solo build the team can still operate and rebuild by the same procedure — a documented hand-off path

Project 9: Observability Hardening — Distributed Tracing & Progressive Delivery

Contribution 100% (Solo design & build — OTel tracing pipeline · Argo Rollouts Canary · soft-launch operation) 2026.06 ~ Present

On top of the AWS EKS production game-launch platform (Project 8), strengthened observability and delivery for the new externally-published game's soft launch.

Filled the missing 'traces' axis of the logging/metrics-only observability platform with end-to-end OpenTelemetry distributed tracing, introduced zero-downtime Canary deployment via Argo Rollouts, and handled real soft-launch traffic on the stack without incident.

9-1. End-to-End Distributed Tracing Pipeline (OTel + Tempo)

Built a distributed tracing stack on AWS EKS: OpenTelemetry Collector + Grafana Tempo (S3 storage via EKS Pod Identity — no static keys) + OpenTelemetry Operator (auto-instrumentation injects language SDKs with no app image change). The key decision was where to sample: RED metrics are generated from 100% of spans before sampling for accuracy, while tail sampling (all errors + everything over 500ms + 10% of the rest) applies only at the trace-storage step to cut storage cost.

Wired Metrics ↔ Traces ↔ Logs into a 3-signal pivot in Grafana, filling the missing 'traces' axis of a logging/metrics-only platform.

9-2. Progressive Delivery with Argo Rollouts (Canary)

The prior Blue-Green swapped the ALB target group all at once on promotion, so healthy targets momentarily dropped to zero and the ALB raced into 503s (observed 2026-07-14). That is why it was switched to Argo Rollouts Canary.

On the same ALB target group, `maxUnavailable: 0` + add-before-remove (register the new Pod before removing the old) lets old and new briefly coexist during the switch, removing the 503 race; the game service is stateless HTTP (state externalized via JWT + Redis/TypeORM), so it stays safe even while traffic is split across old and new. Templated the deployment strategy as an opt-in toggle in the shared base-aws Helm chart and enabled it on just the one production game main service (a sibling internal service keeps a plain Deployment), with the controller running leader-election HA + PDB so rollouts never stall through node drain / Karpenter consolidation.

9-3. Soft-Launch Operation (2026.07 ~ Present)

Went live with the soft launch on 2026-07-14 (KST), opening the new externally-published game to a limited release and handling its small-scale real traffic on the stack above with zero-downtime Canary deploys and 3-signal observability (verified trace_id/span_id correlation in live logs).

HPA · Karpenter autoscaling was provisioned for the 2026 Q4 global launch (no scale-up triggered under soft-launch traffic).

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game and blockchain startup, leading the large-scale cloud migration project from AWS to GCP.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra design/migration lead 70% (remaining 30% = server-team application-side migration collab), CI/CD pipeline 100% 10 months (2023.03 ~ 2023.12)

Problem

Continuously rising AWS costs (\$10,000+/month, high NAT Gateway and data transfer costs), a GCP Startup Program opportunity, and the need for GCP global load balancing for the multi-region game service

Approach

Adopted Shared VPC (Host / Service Project separation) and Workload Identity Federation so GitHub Actions could authenticate to GCP via short-lived tokens without service-account keys. Migrated DNS via subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS) for pre-validation, then performed the final NS swap during a single scheduled maintenance window with minimal downtime.

Worked around Filestore's 2.5TB SSD minimum cost by self-hosting an NFS server on Compute Engine (pd-balanced 1TB). Configured Cloud Armor with region blocking + IP allow-list WAF policy.

Troubleshooting

- During AWS ALB → GCP Global LB transition, routing issues arose because AWS ALB still passes traffic to backends failing health checks whereas GCP LB blocks them.
Reconfigured health checks/backends after analyzing GCP NEG structure
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response.
Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig
- Game client file downloads failed after the CDN migration due to missing HTTP → HTTPS redirect.
Resolved by splitting the URL Map into HTTP / HTTPS variants (`default_url_redirect.https_redirect=true`)

- After DNS delegation to Cloud DNS, the domain was missing from Google Search Console.
Resolved by adding the issued verification TXT record to Cloud DNS to verify ownership, then re-registering

Results

- 50% cloud cost reduction (\$10,000 → \$5,000/month), 50% deployment time reduction, 95% rollback reduction
- Full GCP infrastructure Terraform IaC (TFLint/Checkov static analysis, Terratest unit testing), 3x weekly deployments, 100% deployment history tracking via Git commits
- Minimized downtime with resource-copy migration; final DNS cutover executed during an approximately 2-hour scheduled maintenance window aligned with game service operations
- Eliminated GCP service-account key management overhead and key-leak risk by adopting Workload Identity Federation for GitHub Actions

Project 2: Game Data Collection Automation System

Contribution 100% 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily, with frequent data loss caused by human error

Approach

Loaded multi-source data (MongoDB · CloudSQL · Google Analytics · Dune) into BigQuery — MongoDB via Dataflow ETL, CloudSQL via direct connection, GA/Dune via API — then triggered Daily / Monthly via Cloud Functions + Cloud Scheduler and pushed analyst-facing sheets via the Google Sheets API. A separate Cloud Function periodically deduplicated BigQuery data.

The entire infrastructure (datasets, Cloud Functions, schedulers, storage, Artifact Registry) was managed as Terraform IaC.

Results

- 95% data collection automation and 0% data gap rate by eliminating human error

Project 3: PLG Stack Monitoring System

Contribution 100% 4 months (2024.04 ~ 2024.07)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive responses with excessive time spent on root-cause analysis

Results

- SLI/SLO establishment and Alertmanager alarm noise optimization, proactive detection established, reduced response time, improved availability

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support, designing on-premises cloud infrastructure for financial and public-sector clients and leading operations training.

Key Projects: PaaS Solution · DP Training · Certificate Automation · Ansible Configuration Management

Results

- PaaS solution for 5 clients, stable operation on Kubernetes HA
- Established DP's autonomous Kubernetes operations (6-month training, avg 10 attendees · 4 hrs per session)
- 10x certificate renewal extension (1yr → 10yr), ~50 hours annual operations savings across 10 clients
- Ansible-based server provisioning and configuration management automation achieving zero configuration drift

ETech System

2022.01 ~ 2022.04

System Engineer

Hardware Server and SAN storage construction. Transitioned to DevOps field for career growth.

Opensource Projects

Kubernetes CRD

[k8s-namespace-sync](#) — Automatically sync Kubernetes namespace resources across clusters
[helios-lb](#) — Custom load balancer controller for Kubernetes
[network-policy-generator](#) — Kubernetes NetworkPolicy auto-generation controller

GitHub Actions

[Compress Decompress Action](#) — Compress/decompress files in workflows
[Image Tag Updater](#) — Auto-update image tags in YAML/Helm files
[Go Git Commit Action](#) — Go-based Git commit creation and push
[Extract Commit Action](#) — Extract Git commit SHA, message, and author information
[Multi Git Mirror Action](#) — Automate mirroring across multiple Git repositories
[Kube Diff Action](#) — Compare and report Kubernetes resource changes
[Major Tag Action](#) — Auto-update major version tags to the latest semver release
[Go Changelog Generator](#) — Generate changelogs from Conventional Commits
[Environment Output Setter](#) — Set multiple env vars and workflow outputs with transformations
[Ternary Operator Action](#) — Conditional-logic action evaluating up to 10 conditions
[Contributors Action](#) — Auto-generate a contributors list from repo data
[Kube Events Action](#) — Check K8s cluster events with filtering and PR comments
[Helm Kustomize Lint Action](#) — Lint and validate Helm charts
[Helm OCI Push](#) — Push Helm charts to OCI registries

Ansible Galaxy

[Ansible K8s IAC Tool](#) — Automated K8s and IaC tool installation collection
[Ansible User Management](#) — Linux user and SSH key management role

DevOps Tools

[bash-pilot](#) — AI-powered Bash command assistant CLI tool
[git-bridge](#) — Git repository mirroring and sync CLI tool
[static-file-server](#) — Go-based static file server with directory UI, file preview, search, ZIP batch download
[kube-diff](#) — CLI to diff K8s manifests against a live cluster
[kube-events](#) — CLI to view and summarize K8s events

Chrome Extensions

[Dev Toolkit](#) — Developer utility collection
[QRify](#) — Instant QR code generation for URLs/text

Education

Chungnam National University - Public Administration (Transfer)	2018.03 ~ 2020.08
National Institute for Lifelong Education - Business Administration (Academic Credit Bank)	2016.09 ~ 2018.02

Certifications

Engineer Information Processing	2021.11
AWS Certified Solution Architect - Associate (Expired)	2021.11
Network Administrator Level 2	2021.10
Linux Master Level 2	2021.10
Computer Specialist Level 1	2021.04
TOEIC Speaking Test - 130/Intermediate Mid 3 (Expired)	2021.02

Study

Tech blog (somaz.blog, English) — DevOps · Kubernetes · IaC writing for global readers	2025.01 ~ 현재
Tech blog (somaz.tistory.com, Korean) — ongoing DevOps · Kubernetes · IaC learning and operations notes	2023.04 ~ 현재
T101(Terraform) Study	2023.08 ~ 2023.10
AEWS(EKS) Study	2023.04 ~ 2023.06

PKOS(Kubernetes) Study

Cloud Security Training

Security Solution Operation Specialist Training Program

2023.01 ~ 2023.02

2022.02 ~ 2022.04

2021.07 ~ 2021.12